

Person 1 – Advocate Backend Developer

Objective

Build the complete backend for advocate onboarding, profile management, verification, dashboard, and consultation management. Your code lives within the `api` monolith, in a new `advocate` module, plus supporting work in `identity` and `trust` modules when needed.

Pre-requisites

- API contracts document (`docs/api-contracts-v1.md`) reviewed and signed off by the team.
- Phase 0 architecture: monolith `api` with domain modules (`identity` , `advocate` , `matching` , `trust` , `conversation`). The `advocate` module's directory `api/src/advocate/` already exists or you will create it.
- `identity` module already provides JWT generation and verification middleware (`requireAdvocate`). If not, coordinate with Person 2 (who implements citizen auth) to reuse the same JWT secret and middleware.

Scope of Work

You own **all advocate-facing REST endpoints**, no citizen- or AI-specific logic. You also own the advocate-related database tables and the admin verification queue.

Tasks (in order)

T1. Advocate Database Schema

- Design and create migration(s) for `advocates` table and related tables (`advocate_profiles` , `advocate_verification_documents` , `advocate_availability` if needed).
- Include fields: `id` , `user_id` (link to `users` table built by Person 2), `bar_enrolment_number` (unique), `state_bar` , `verification_status` (enum: `pending` , `verified` , `rejected`), `profile` (JSONB for the six 2008-permitted fields: name, address, phone, email, practice areas, courts), `languages` (array of text), `districts` (array of text), `years_of_practice` , `certificate_of_practice_url` (S3 key), `created_at` , `updated_at` .

- Create a separate `verification_documents` table for uploaded CoP and other proof, linked to `advocate_id`.
- Reference: the `advocate` object shape from the matching response in the contract.

T2. Advocate Registration & Onboarding Endpoints

Implement the following API endpoints inside `api/src/advocate/advocate.controller.ts` (or whatever framework you use) and the corresponding service.

- `POST /api/advocate/register`
Request: `{ phone: string }` (already OTP-verified earlier? Or combined with auth). For simplicity, assume advocate sign-up uses the same OTP flow as citizens (Person 2 provides `/api/auth/request-otp` and `/api/auth/verify-otp` that also work for advocates). After OTP verification, a user record is created; then this endpoint creates the advocate-specific record.
Response: `{ advocateId, ... }` with a JWT token that has advocate role.
- `PUT /api/advocate/profile`
Request: the six-field profile object (name, address, phone, email, practice areas, courts). Validates that fields comply with the 2008 BCI rules.
- `POST /api/advocate/documents`
Multipart upload for Certificate of Practice and any additional verification documents. Stores file in S3/MinIO, creates a record in `verification_documents`.
- `PUT /api/advocate/availability`
Request: `{ available: boolean, schedule?: { ... } }` (optional for Phase 1 scheduling).
- `POST /api/advocate/submit-verification`
Changes advocate status from `pending` to `pending` (actually triggers admin review). No body.

T3. Advocate Dashboard Endpoints

- `GET /api/advocate/dashboard`
Returns summary: total consultations requested/accepted/completed, earnings (if any), profile completeness score, unread messages count (the advocate side of the conversation).
- `GET /api/advocate/consultations`
Returns list of consultation requests (`status: requested`) and ongoing (`status: accepted`) for this advocate, with minimal matter details (matter type, citizen's anonymised ID).
- `GET /api/advocate/consultations/{consultationId}`
Full consultation detail, including matter information (citizen's query and AI response) only after the consultation is accepted (or if the advocate needs to review before accepting).

- `PUT /api/advocate/consultations/{consultationId}`

Accept or decline: `{ action: "accept" | "decline", declineReason?: string }`. This triggers the status change that Person 2's WebSocket server uses to open the chat.

T4. Admin Verification Queue

- `GET /api/admin/advocates/pending` (requires admin role)
Returns list of advocates with status `pending`, along with uploaded documents.
- `PUT /api/admin/advocates/{advocateId}/verify`
`{ action: "approve" | "reject", reason?: string }`. Updates verification status, sends appropriate notification (optional).
- Use the existing `identity` module's role-based middleware (`requireAdmin`).

T5. Integration Points and Testing

- Ensure that the `matching` module (Person 2) can query your advocates table to find matches.
Agree on the read-only views or a service method you expose.
- Test all endpoints with a tool like Postman against a seeded database.
- Coordinate with Person 2 on the shared `consultations` table (you may have to create it, or Person 2 creates it). Decide ownership: probably Person 2 creates `consultations` and `messages` tables, but you need to update consultation status from your endpoint.

Deliverables

- All endpoints listed above, responding correctly as per the agreed contracts (the advocate part of the contracts, plus any internal ones you defined).
- Database migration files for advocate tables.
- At least 5 manual test cases (one for each flow) that Person 3 (or a reviewer) can run.
- Updated `docs/api-contracts-v1.md` with any corrections that emerged while building.

Person 2 – Backend/AI Developer (Citizen System & AI Pipeline)

Objective

Build the entire citizen-facing backend: anonymous intake, AI integration, matter management, session handling, user authentication, document upload, consultation request, and the WebSocket chat server.

You also build the separate AI service (Python) that provides the three core endpoints (classify, retrieve, generate).

Pre-requisites

- API contracts document frozen.
- Phase 0 architecture: monolith `api` (Node.js) with modular folders; separate AI service (Python, e.g., FastAPI) that runs in its own container.
- Shared database (Postgres) and Redis (for session/OTP/presence if needed) are available.

Scope of Work

- `identity` module: user sign-up/sign-in via phone OTP (works for both citizens and advocates, but you'll implement the citizen flow first).
- `matter` module: anonymous matter creation, retrieval, listing, document association.
- `matching` module: advocate shortlisting logic (queries advocate table built by Person 1).
- `trust` module: audit logging and Rule 36 moderation hook (used later in chat, but scaffold it).
- `conversation` module: WebSocket server for matter-scoped chat, message persistence.
- AI service: three endpoints, corpus ingestion, LLM integration.

Tasks (in order)

T1. Database & Session Infrastructure

- Create migrations for `users`, `matters`, `consultations`, `messages`, `documents` tables.
- `users`: `id`, `phone`, `phone_verified`, `role` (citizen/advocate/admin), `preferred_language`.
- `matters`: `id`, `session_id` (nullable, for anonymous), `user_id` (nullable, set after claim), `query_text`, `query_language`, `classification` (JSONB), `citations` (JSONB), `ai_response` (JSONB), `status` (session-owned, user-owned, archived), `created_at`, `expires_at`.
- `consultations`: `id`, `matter_id`, `citizen_id`, `advocate_id`, `status` (requested/accepted/declined/closed), `requested_at`, `accepted_at`.
- `messages`: `id`, `consultation_id`, `sender_type` (citizen/advocate), `sender_id`, `content`, `moderation_status` (approved/pending/flagged), `created_at`.
- `documents`: `id`, `matter_id`, `uploader_id`, `file_path`, `file_type`, `size`, `scan_status`.

- Implement session management: when an anonymous request arrives, generate a session ID (UUID), store minimal session info in Redis with TTL 24h, set cookie. On sign-up, load session's matters and assign to user.

T2. AI Service (Python)

Create a new project in `services/ai/` with requirements.txt (FastAPI, httpx, langchain, openai, etc.). Implement three endpoints as defined in the contracts:

- `POST /classify`: input { `query`, `language` }, output { `matter_type`, `statute`, `user_question`, `involves_police`, `incident_date`, `location` }. Use an LLM call with a strict schema.
- `POST /retrieve`: input { `matter_type`, `statute`, `jurisdiction` }, output { `citations`: [...] }. Use a vector database or simple keyword search against pre-ingested corpuses (India Code, WB Acts, Calcutta HC judgments). For Phase 1, a small manually curated lookup or simple BM25 is acceptable.
- `POST /generate`: input { `query`, `citations`, `language` }, output { `english_text`, `bengali_text`, `disclaimer` }. Use an LLM with a system prompt that strictly grounds the response in the provided citations and appends the disclaimer.
- Note: Bengali translation can be done by a separate model or the same LLM (if capable). For Phase 1, you may use a single LLM that outputs both languages in one call.

Corpus ingestion: Write a script that downloads and chunks central acts (India Code, WB acts), and a selection of Calcutta HC judgments (from AWS open data). Store chunk embeddings or plain text in a searchable format (SQLite/FTS, or Postgres full-text). This can be done once manually; the retrieval endpoint queries this store.

T3. Citizen Backend Endpoints (in `api/src/` under appropriate modules)

Implement the REST endpoints defined in the contract document:

- `POST /api/matter` (anonymous) – the main intake flow. Call AI service sequentially (classify → retrieve → generate), store matter, run matching, return.
- `GET /api/matter/{matterId}` – retrieve matter for session or user.
- `GET /api/matter/{matterId}/advocates` – paginated advocate list.
- `POST /api/auth/request-otp` – validate phone, store OTP in Redis, call external SMS service (or log to console for dev).
- `POST /api/auth/verify-otp` – verify OTP, create/find user, issue JWT, claim session matters.
- `POST /api/matter/{matterId}/documents` – handle file upload, scan, link to matter.
- `POST /api/consultations` – create consultation record, set status `requested`.

T4. WebSocket Chat Server

- Inside `api/src/conversation/`, implement a WebSocket server (using `ws` or `socket.io`).
- On connection, verify JWT, check user is participant of the consultation.
- Handle incoming messages: persist to `messages` table, pass advocate messages through the moderation hook (regex-based + optionally LLM call) before broadcasting to citizen.
- Handle delivery receipts, typing indicators (optional).
- Ensure audit log entries for every message (in `trust` module).

T5. Matching Logic

- In `matching` module, create a function `matchAdvocates(matterClassification, limit)` that queries the `advocates` table (from Person 1's schema) filtering by practice area overlap, district, language, verification status (verified first). Use Postgres query. Person 1 must give you read access to those tables.

T6. Rule 36 Moderation Hook

- In `trust` module, create a function that accepts an advocate message string and returns `{ status: "approved" | "flagged", reason? }`. Use a list of forbidden phrases, maybe a small LLM classifier. For Phase 1, regex is enough.

Deliverables

- All citizen REST endpoints fully functional.
- AI service running as a separate process, responding correctly to the three internal calls.
- WebSocket chat server working end-to-end (test with a simple HTML client).
- Database migrations for all tables you own.
- Integration points with Person 1's advocate table proven (mock data if needed).
- Documentation of the AI service API and how to run it.

Person 3 – Frontend Developer (Citizen UI)

Objective

Build the entire citizen-facing web application using Next.js (app router). From the anonymous intake page through AI response display, advocate matching, sign-up flow, document upload, and the real-time chat with advocates. The UI must be bilingual (English/Bengali) and responsive (mobile-first).

Pre-requisites

- API contracts frozen and shared.
- Backend endpoints stubbed or running locally (Person 2 will provide you with a running dev instance, but you can start with mock data).
- Design tokens and shared UI components from `shared/` if any (or you will create a simple design system).

Scope of Work

- Pages(routes): `/` (landing), `/app/intake` (anonymous query), `/app/matter/[id]` (AI response + advocates), `/auth/signup` (OTP flow), `/app/matters` (user's saved matters when logged in), `/app/chat/[consultationId]` (real-time chat).
- State management for anonymous session, user authentication, and matter data.
- WebSocket client integration for chat.
- Bilingual interface: implement language switching (En/Bn) and ensure all static strings are translated; the AI response already contains both languages.

Tasks (in order)

T1. Project Setup & Routing

- Inside `apps/frontend/web/`, set up Next.js app router.
- Create layout with language switcher.
- Implement route guards: `/app/*` pages check if user has a valid JWT (or redirect to signup). The intake page and `/app/matter/[id]` are accessible without login but require the session cookie (handled automatically by browser).
- Set up API client utility (e.g., using fetch/axios) that automatically includes session cookie for anonymous endpoints and Authorization header for authenticated ones.

T2. Anonymous Intake Page (`/app/intake`)

- A simple, large text input (like a chat box) for the user to describe their legal problem. A language selector toggle above the input (English | বাংলা).
- On submit, send `POST /api/matter` with the text and chosen language.
- Handle loading state (show "Analysing your situation..." with a progress animation), error states, and success.
- On success, redirect to `/app/matter/{matterId}`.

T3. Matter Response & Advocate Preview Page (`/app/matter/[id]`)

- Fetch matter data from `GET /api/matter/{matterId}` (or use data from the POST redirect, but implement fetching for refresh).
- Display:
 - **Classification summary** (matter type, possible statute – optional).
 - **AI Response** section: a tabbed panel or side-by-side with English and Bengali responses, each with a clear disclaimer at the end. Highlight citations (clickable links if URLs present).
 - **Matching Advocates** section: card list showing name, enrolment number, practice areas, languages, districts, and a "Verified" badge. No contact info yet. A "Request Consultation" button next to each (disabled if not signed in; if tapped, prompt sign-up flow).
- If the user is not signed in, show a CTA: "Save this matter and connect with an advocate – Sign up now."
- If session expires, show appropriate message.

T4. Sign-Up Flow (`/auth/signup` and embedded OTP modal)

- Build a two-step form:
 1. Enter phone number → call `POST /api/auth/request-otp`.
 2. Enter OTP → call `POST /api/auth/verify-otp` (automatically claims session matters).
- On success, store the JWT (in memory or a secure httpOnly cookie? For simplicity, store in `localStorage` or a cookie, but ensure it's sent with requests).
- After sign-up, redirect back to the matter they were viewing (or to `/app/matters`).

T5. Document Upload (on Matter page, when signed in)

- Add a file upload button to the matter detail page. Use `POST /api/matter/{matterId}/documents` as multipart form.
- After upload, show the document in a list with file name and size. Click to view (if possible) or download.

T6. Consultation Request & Real-Time Chat

- After user signs in and clicks "Request Consultation" on an advocate card, call `POST /api/consultations` with `matterId` and `advocateId`.
- Show consultation status: "Request sent. Waiting for advocate to accept." Poll the consultation endpoint or listen for a server-sent event? For simplicity, poll `GET /api/consultations/{consultationId}` every few seconds until status becomes `accepted`.
- Once accepted, enable a "Chat" button that navigates to `/app/chat/[consultationId]`.
- On the chat page, establish a WebSocket connection to `ws://localhost:3000/ws/consultation/{consultationId}?token=<jwt>`.

Display conversation history (fetch via `GET /api/matter/{matterId}/messages` ? Wait, the contracts don't include that; you might need to add an endpoint. Negotiate with Person 2 to add `GET /api/consultations/{consultationId}/messages` as a fallback for initial load). Initially show messages loaded from the REST endpoint, then live messages from WebSocket.

- Implement message composer, send via WebSocket, display sent messages with moderation status (if advocate message is pending moderation, show a subtle indicator to advocate? For citizen, only show approved messages).
- Handle connection loss and reconnection.

T7. Bilingual Support

- Create a translation file for all UI strings (English/Bengali). Use a context or i18n library.
- Ensure language toggling persists across pages (localStorage or cookie). The AI response will always come in both languages; you can show one or both based on user preference.
- Test with Bengali text: the intake page must handle Bengali input correctly (font, text direction).

Deliverables

- All citizen pages fully functional and responsive.
 - Integration with all backend endpoints as per contracts.
 - WebSocket chat working end-to-end.
 - A simple toggleable bilingual interface.
 - Two demo user journeys recorded (video or screenshots): one anonymous intake → AI response, and one full sign-up → consultation request → chat.
-

Integration and Communication Plan

- **Shared repository:** both backend code lives in `legallink-be` (monolith). Person 1 works in `src/advocate/`, Person 2 in `src/identity/`, `src/matter/`, `src/matching/`, `src/trust/`, `src/conversation/`. They must agree on shared code for database connection, middleware, and error handling. Schedule a brief daily sync to resolve merge conflicts.
- **Database:** Person 2 creates the initial migration with shared tables (`users`). Person 1 later adds migrations for `advocates` and related tables. Person 2's `matching` module needs read access to `advocates` and `users` — Person 1 will provide a service or repository function.
- **Testing:** Person 3 will need Person 2's endpoints running early. Person 2 should provide a Postman collection or a deployed dev instance. Person 3 can start with mock data, but final integration tests require both backends up.

- **Chat server:** Person 2 owns the WebSocket server. Person 1 just ensures advocate consultation status changes trigger no special logic (the chat server just checks consultation status on connection).